Volledige cheatsheet – C# Web (week 1 t/m 7)

Dit is het naslagwerk met **alle leerstof** uit de planning. Voor de PE zelf (Oefening 15-stijl) gebruik je eerst `Cheatsheet-PE.pdf`. Deze PDF gebruik je als iets gevraagd wordt dat daar niet in staat.

#	Onderwerp	Week
1	Empty Web Application	1
2	Razor-syntax	1
3	Razor Pages	1
4	Middleware	1
5	MVC: mappen, routing, action results	2
6	Data naar de view + ViewModel + paging	2
7	Model Validation	3
8	Entity Framework: opzet + CRUD	3–4
9	Volledige CRUD-controller (Index, Details, Create, Edit, Delete)	3–5
10	Relaties (1-n, n-m, 1-1) + Include	4
11	EF Configuration (annotaties + Fluent API)	4
12	EF Tools (migrations, Scaffold-DbContext)	5
13	Scaffolding in Visual Studio	3, 5
14	Partial View	5
15	TagHelpers (eigen + ingebouwde)	6
16	ViewComponents	6–7
17	LINQ queries	7
18	Formulieren: dropdown, enum, checkbox, datum	–

1. Empty Web Application

Project: *ASP.NET Core Empty*. Alles staat in `Program.cs`.

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.MapGet("/", () => "Hello World!");
app.MapGet("/hallo/{naam}", (string naam) => $"Hallo {naam}");      // route-parameter
app.MapGet("/som", (int a, int b) => a + b);                         // /som?a=2&b=3 (query string)
app.MapGet("/html", () => Results.Content("<h1>Hallo</h1>", "text/html"));

app.Run();
```

Statische bestanden (HTML, CSS en afbeeldingen in `wwwroot`):

```
app.UseDefaultFiles();   // zoekt wwwroot/index.html als je naar "/" surft
app.UseStaticFiles();    // geeft bestanden uit wwwroot terug
```

`UseDefaultFiles` moet **vóór** `UseStaticFiles` staan.

2. Razor-syntax (in .cshtml)

Wat	Code
Waarde tonen	`@Model.Merk` of `@DateTime.Now.Year`
Expressie tonen	`@(Model.Km * 2)` of `@(a ? "ja" : "nee")`
Codeblok	`@{ var totaal = Model.Count(); }`
If	`@if (Model.Any()) { <p>...</p> } else { <p>Leeg</p> }`
Lus	`@foreach (var v in Model) { <li>@v.Merk</li> }`
For	`@for (int i = 1; i <= 5; i++) { <span>@i</span> }`
Commentaar	`@* dit wordt niet getoond *@`
HTML niet escapen	`@Html.Raw("<b>vet</b>")`
Tekst in een codeblok	`<text>gewone tekst</text>` of `@:gewone tekst`
Model-type	`@model Voertuig` / `@model IEnumerable<Voertuig>`
Using	`@using MVCVoertuig.ViewModels`
Service injecteren	`@inject VoertuigDbContext Db`
Bedrag / getal / datum	`@x.ToString("C")` / `@x.ToString("N0")` / `@d.ToString("dd/MM/yyyy")`

Speciale bestanden

- `Views/_ViewStart.cshtml` → `@{ Layout = "_Layout"; }` (layout voor elke view)
- `Views/_ViewImports.cshtml` → `@using` en `@addTagHelper` voor **alle** views
- `Views/Shared/_Layout.cshtml` → het kader rond elke pagina: `@RenderBody()`
- **Sections** in de layout: `@await RenderSectionAsync("Scripts", required: false)`. In een view vul je die met `@section Scripts { ... }`.
- Geen layout voor één view: `@{ Layout = null; }`

---

3. Razor Pages

Program.cs:

```
builder.Services.AddRazorPages();
// ...
app.MapRazorPages();
```

Map `Pages/`. De URL volgt het pad: `Pages/Voertuigen/Index.cshtml` wordt `/Voertuigen`.

⚠️ De map `Pages/` heeft een **eigen** `Pages/_ViewImports.cshtml` nodig, anders werken `asp-for` en `asp-page` niet (ze staan dan letterlijk in de HTML):

```
@using MVCVoertuig
@using MVCVoertuig.Models
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

Voor de layout zet je in `Pages/_ViewStart.cshtml`: `@{ Layout = "_Layout"; }`. De layout zelf mag in `Pages/Shared/` of `Views/Shared/` staan.

`Pages/Voertuigen/Create.cshtml`

```
@page
@model CreateModel

<form method="post">
    <input asp-for="Voertuig.Merk" class="form-control" />
    <span asp-validation-for="Voertuig.Merk" class="text-danger"></span>
    <button type="submit" class="btn btn-primary">Opslaan</button>
</form>
<a asp-page="./Index">Terug</a>
```

`Pages/Voertuigen/Create.cshtml.cs`

```cs
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;

public class CreateModel : PageModel
{
    private readonly VoertuigDbContext _context;
    public CreateModel(VoertuigDbContext context) { _context = context; }

    [BindProperty]                          // vult de property met de formulierwaarden bij POST
    public Voertuig Voertuig { get; set; } = new();

    public void OnGet() { }                 // GET-request

    public IActionResult OnPost()           // POST-request
    {
        if (!ModelState.IsValid) return Page(); // zelfde pagina opnieuw tonen
        _context.Voertuigen.Add(Voertuig);
        _context.SaveChanges();
        return RedirectToPage("./Index");
    }
}
```

Razor Pages	MVC-equivalent
`OnGet()` / `OnPost()` / `OnGetAsync()`	actie met `[HttpGet]` / `[HttpPost]`
`return Page();`	`return View();`
`RedirectToPage("./Index")`	`RedirectToAction("Index")`
`asp-page="/Voertuigen/Index"`	`asp-controller` + `asp-action`
`@page "{id:int}"` + `OnGet(int id)`	`/Details/5`
`[BindProperty]`	parameter van de POST-actie
meerdere POST-handlers: `OnPostDelete()` + `<button asp-page-handler="Delete">`	aparte acties

Een lijst tonen gaat met een property in de PageModel, bv. `public List<Voertuig> Voertuigen { get; set; } = new();`. Vul die in `OnGet`. In de view lees je ze dan als `@Model.Voertuigen`.

---

4. Middleware

Middleware is een rij stappen waar **elke request** in volgorde door gaat. Elke stap kan iets doen vóór en na de volgende stap, of de request afbreken.

```cs
// Inline middleware: gaat verder naar de volgende stap
app.Use(async (context, next) =>
{
    Console.WriteLine($"Request: {context.Request.Path}");   // VOOR de volgende
    await next(context);                                     // naar de volgende middleware
    Console.WriteLine($"Status: {context.Response.StatusCode}"); // NA de volgende
});

// Tak voor een pad: alles onder /info komt hier
app.Map("/info", info =>
{
    info.Run(async context => await context.Response.WriteAsync("Info-pagina"));
});

// Tak op voorwaarde
app.MapWhen(context => context.Request.Query.ContainsKey("debug"),
    debug => debug.Run(async context => await context.Response.WriteAsync("Debug-modus")));

// Run = EINDPUNT: hierna wordt niets meer uitgevoerd
app.Run(async context => await context.Response.WriteAsync("Einde van de pipeline"));
```

Eigen middleware-klasse

```
public class LogMiddleware
{
    private readonly RequestDelegate _next;
    public LogMiddleware(RequestDelegate next) { _next = next; }

    public async Task InvokeAsync(HttpContext context)
    {
        var start = DateTime.Now;
        await _next(context);
        var duur = DateTime.Now - start;
        Console.WriteLine($"{context.Request.Path} duurde {duur.TotalMilliseconds} ms");
    }
}

// Extension-methode (optioneel) zodat je app.UseLog() kunt schrijven
public static class LogMiddlewareExtensions
{
    public static IApplicationBuilder UseLog(this IApplicationBuilder app)
        => app.UseMiddleware<LogMiddleware>();
}
```

In Program.cs: `app.UseMiddleware<LogMiddleware>();` of `app.UseLog();`

Volgorde (fout = vreemde fouten): `UseExceptionHandler` → `UseHsts` → `UseHttpsRedirection` → `UseStaticFiles` / `MapStaticAssets` → `UseRouting` → `UseAuthentication` → `UseAuthorization` → `Map...` → `Run`.

Handig in middleware: `context.Request.Path`, `context.Request.Method`, `context.Request.Query["x"]`, `context.Response.StatusCode = 404;`, `context.Response.Redirect("/");`, `context.Response.ContentType = "text/html";`

---

5. MVC: mappen, routing en action results

Program.cs: `builder.Services.AddControllersWithViews();` + `app.MapControllerRoute(...)`

Wat	Waar
`VoertuigController`	`Controllers/VoertuigController.cs` (naam **eindigt** op Controller)
View van actie `Index`	`Views/Voertuig/Index.cshtml`
Gedeelde views, layout, partials	`Views/Shared/`
Model	`Models/`; ViewModel
CSS, JS en afbeeldingen	`wwwroot/` (in HTML: `~/css/site.css`)

Conventionele route: `pattern: "{controller=Home}/{action=Index}/{id?}"`

- `/Voertuig/Details/5` → `VoertuigController.Details(int id)` met id = 5
- `/Voertuig?merk=BMW` → `Index(string merk)` (query string)

Extra route (die staat **vóór** de default):

```
app.MapControllerRoute(name: "merk", pattern: "merk/{merk}", defaults: new { controller = "Voertuig", action = "Index"
});
```

Attribute routing:

```
[Route("voertuigen")]
public class VoertuigController : Controller
{
    [HttpGet("")]                public IActionResult Index() => View();
    [HttpGet("detail/{id:int}")] public IActionResult Details(int id) => View();
}
```

Action results | Code | Resultaat | |---|---| | `View()` / `View(model)` / `View("AndereView", model)` | view tonen | | `RedirectToAction("Index")` / `RedirectToAction("Index", "Home")` | naar een andere actie | | `RedirectToAction("Details", new { id = 5 })` | met parameter | | `NotFound()` / `BadRequest()` / `Ok()` | statuscode 404 / 400 / 200 | | `Content("tekst")` / `Json(object)` | tekst / JSON | | `PartialView("_X", model)` / `ViewComponent("HeaderMenu")` | alleen een stukje | | `Redirect("https://...")` | naar een URL |

Links in views: `<a asp-controller="Voertuig" asp-action="Details" asp-route-id="@v.VoertuigId">Details</a>`. Elke `asp-route-xxx` wordt een parameter `xxx`.

---

6. Data naar de view + ViewModel + paging

Manier	Controller	View	Opmerking
Model	`return View(lijst);`	`@model ...` → `@Model`	sterk getypeerd (beste keuze)
ViewBag	`ViewBag.Titel = "x";`	`@ViewBag.Titel`	dynamisch
ViewData	`ViewData["Titel"] = "x";`	`@ViewData["Titel"]`	woordenboek
TempData	`TempData["Melding"] = "Opgeslagen!";`	`@TempData["Melding"]`	overleeft **één** redirect

ViewModel = een klasse speciaal voor één view, die meerdere dingen samenbundelt.

```
namespace MVCVoertuig.ViewModels
{
    public class PagingInfo
    {
        public int TotalItems { get; set; }
        public int ItemsPerPage { get; set; }
        public int CurrentPage { get; set; }
        public int TotalPages => (int)Math.Ceiling((decimal)TotalItems / ItemsPerPage);
    }

    public class VoertuigOverzichtViewModel
    {
        public IEnumerable<Voertuig> Voertuigen { get; set; } = new List<Voertuig>();
        public PagingInfo PagingInfo { get; set; } = new();
        public string? HuidigMerk { get; set; }
    }
}
```

Controller met paging:

```
public int PageSize = 3;

public IActionResult Index(string? merk, int pagina = 1)
{
    var query = _context.Voertuigen
        .Where(v => merk == null || v.Merk == merk)
        .OrderBy(v => v.VoertuigId);

    var model = new VoertuigOverzichtViewModel
    {
        Voertuigen = query.Skip((pagina - 1) * PageSize).Take(PageSize).ToList(),
        PagingInfo = new PagingInfo { CurrentPage = pagina, ItemsPerPage = PageSize, TotalItems = query.Count() },
        HuidigMerk = merk
    };
    return View(model);
}
```

View:

```
@model VoertuigOverzichtViewModel

@foreach (var v in Model.Voertuigen)
{
    <partial name="_VoertuigCard" model="v" />
}

<ul class="pagination">
    @for (int i = 1; i <= Model.PagingInfo.TotalPages; i++)
    {
        <li class="page-item @(i == Model.PagingInfo.CurrentPage ? "active" : "")">
            <a class="page-link" asp-action="Index" asp-route-pagina="@i" asp-route-merk="@Model.HuidigMerk">@i</a>
        </li>
    }
</ul>
```

(Zet `@using MVCVoertuig.ViewModels` in `_ViewImports.cshtml`.)

7. Model Validation

Alle annotaties (`using System.ComponentModel.DataAnnotations;`)

Annotatie	Betekenis
`[Required]`	verplicht
`[StringLength(100)]` / `[StringLength(100, MinimumLength = 2)]`	max (en min) aantal tekens
`[MaxLength(100)]` / `[MinLength(2)]`	max / min lengte
`[Range(0, 100)]` / `[Range(typeof(decimal), "0", "9999.99")]`	tussen 2 waarden
`[RegularExpression(@"^[A-Z]{2}\d{4}$")]`	patroon
`[EmailAddress]` / `[Phone]` / `[Url]` / `[CreditCard]`	formaat
`[Compare("Wachtwoord")]`	moet gelijk zijn aan een ander veld
`[DataType(DataType.Date)]`	alleen datum (input type=date)
`[DataType(DataType.Password)]` / `DataType.MultilineText`	wachtwoordveld / textarea
`[DataType(DataType.Currency)]`	geldbedrag
`[Display(Name = "Verkoopprijs")]`	label-tekst
`[DisplayFormat(DataFormatString = "{0:C}")]`	weergave-formaat

Foutmeldingen met placeholders:

```
[Required(ErrorMessage = "{0} is verplicht")]                                    // {0} = veldnaam
[StringLength(50, MinimumLength = 2, ErrorMessage = "{0}: tussen {2} en {1} tekens")] // {1} = max, {2} = min
[Range(1, 10, ErrorMessage = "{0} moet tussen {1} en {2} liggen")]               // {1} = min, {2} = max
```

In de controller

```
if (voertuig.Merk == "TEST")
    ModelState.AddModelError("Merk", "Dit merk mag niet");   // fout bij een veld
ModelState.AddModelError("", "Algemene fout");               // verschijnt in validation-summary

if (!ModelState.IsValid) return View(voertuig);
```

In de view

```
<div asp-validation-summary="ModelOnly" class="text-danger"></div>  @* All = ook veldfouten *@
<span asp-validation-for="Merk" class="text-danger"></span>

@section Scripts { @{ await Html.RenderPartialAsync("_ValidationScriptsPartial"); } }  @* client-side *@
```

Custom validatie-attribuut

```
public class BouwjaarAttribute : ValidationAttribute
{
    protected override ValidationResult? IsValid(object? value, ValidationContext validationContext)
    {
        int jaar = Convert.ToInt32(value);
        if (jaar < 1900 || jaar > DateTime.Now.Year)
            return new ValidationResult(ErrorMessage ?? $"Jaar moet tussen 1900 en {DateTime.Now.Year} liggen");
        return ValidationResult.Success;
    }
}
// Gebruik: [Bouwjaar] of [Bouwjaar(ErrorMessage = "Eigen tekst")]
```

Een ander veld van hetzelfde object lezen (bv. de einddatum moet na de startdatum liggen):

```
public class NaStartDatumAttribute : ValidationAttribute
{
    protected override ValidationResult? IsValid(object? value, ValidationContext validationContext)
    {
        var huur = (Huur)validationContext.ObjectInstance;       // het volledige object
        if (value is DateTime eind && eind <= huur.StartDatum)
            return new ValidationResult("Einddatum moet na de startdatum liggen");
        return ValidationResult.Success;
    }
}
```

IValidatableObject (validatie over meerdere velden, in het model zelf)

```
public class Huur : IValidatableObject
{
    public DateTime StartDatum { get; set; }
    public DateTime EindDatum { get; set; }

    public IEnumerable<ValidationResult> Validate(ValidationContext validationContext)
    {
        if (EindDatum <= StartDatum)
            yield return new ValidationResult("Einddatum moet na de startdatum liggen", new[] { nameof(EindDatum) });
    }
}
```

Custom en IValidatableObject werken alleen **server-side**: de fout verschijnt pas na het submitten.

---

8. Entity Framework: opzet en CRUD

Packages: `Microsoft.EntityFrameworkCore.SqlServer` + `Microsoft.EntityFrameworkCore.Tools`

DbContext → `Data/VoertuigDbContext.cs`

```
public class VoertuigDbContext : DbContext
{
    public VoertuigDbContext(DbContextOptions<VoertuigDbContext> options) : base(options) { }
    public DbSet<Voertuig> Voertuigen { get; set; } = null!;
}
```

appsettings.json: `"ConnectionStrings": { "VoertuigConnection": "Server=(localdb)\\MSSQLLocalDB;Database=VoertuigDb;Trusted_Connection=True;MultipleActiveResultSets=true" }`

Program.cs:

```
builder.Services.AddDbContext<VoertuigDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("VoertuigConnection")));
```

Conventies: een property `Id` of `<Klasse>Id` wordt de primary key (identity). `<Klasse>Id` + navigation property wordt een foreign key. De tabelnaam is de naam van de DbSet.

CRUD

Wat	Sync	Async (`using Microsoft.EntityFrameworkCore;`)
Alles	`_context.Voertuigen.ToList()`	`await _context.Voertuigen.ToListAsync()`
Op primary key	`_context.Voertuigen.Find(id)`	`await _context.Voertuigen.FindAsync(id)`
Eerste die voldoet	`.FirstOrDefault(v => v.VoertuigId == id)`	`await ....FirstOrDefaultAsync(...)`
Toevoegen	`_context.Voertuigen.Add(v);`	`await _context.Voertuigen.AddAsync(v);`
Wijzigen	`_context.Voertuigen.Update(v);`	–
Verwijderen	`_context.Voertuigen.Remove(v);`	–
Opslaan (**altijd na Add/Update/Remove!**)	`_context.SaveChanges();`	`await _context.SaveChangesAsync();`
Bestaat?	`_context.Voertuigen.Any(v => v.VoertuigId == id)`	`await ....AnyAsync(...)`
Alleen lezen (sneller)	`_context.Voertuigen.AsNoTracking().ToList()`	

Seed data bij het opstarten: zie Cheatsheet-PE stap 10 (`SeedData.EnsurePopulated(app)`). Je kunt ook `HasData` gebruiken, zie hoofdstuk 11.

---

9. Volledige CRUD-controller

Dit is wat *Scaffolding* genereert, maar dan korter. Gebruik het als je **Details, Edit of Delete** zelf moet schrijven.

```
public class VoertuigController : Controller
{
    private readonly VoertuigDbContext _context;
    public VoertuigController(VoertuigDbContext context) { _context = context; }

    public async Task<IActionResult> Index()
        => View(await _context.Voertuigen.ToListAsync());

    // GET /Voertuig/Details/5
    public async Task<IActionResult> Details(int? id)
    {
        if (id == null) return NotFound();
        var voertuig = await _context.Voertuigen.FirstOrDefaultAsync(v => v.VoertuigId == id);
        if (voertuig == null) return NotFound();
        return View(voertuig);
    }

    public IActionResult Create() => View();

    [HttpPost, ValidateAntiForgeryToken]
    public async Task<IActionResult> Create(Voertuig voertuig)
    {
        if (!ModelState.IsValid) return View(voertuig);
        _context.Add(voertuig);
        await _context.SaveChangesAsync();
        return RedirectToAction(nameof(Index));
    }

    // GET /Voertuig/Edit/5
    public async Task<IActionResult> Edit(int? id)
    {
        if (id == null) return NotFound();
        var voertuig = await _context.Voertuigen.FindAsync(id);
        if (voertuig == null) return NotFound();
        return View(voertuig);
    }

    [HttpPost, ValidateAntiForgeryToken]
    public async Task<IActionResult> Edit(int id, Voertuig voertuig)
    {
        if (id != voertuig.VoertuigId) return NotFound();
        if (!ModelState.IsValid) return View(voertuig);
        _context.Update(voertuig);
        await _context.SaveChangesAsync();
        return RedirectToAction(nameof(Index));
    }

    // GET /Voertuig/Delete/5 -> bevestigingspagina
    public async Task<IActionResult> Delete(int? id)
    {
        if (id == null) return NotFound();
        var voertuig = await _context.Voertuigen.FirstOrDefaultAsync(v => v.VoertuigId == id);
        if (voertuig == null) return NotFound();
        return View(voertuig);
    }

    [HttpPost, ActionName("Delete"), ValidateAntiForgeryToken]
    public async Task<IActionResult> DeleteConfirmed(int id)
    {
        var voertuig = await _context.Voertuigen.FindAsync(id);
        if (voertuig != null) _context.Voertuigen.Remove(voertuig);
        await _context.SaveChangesAsync();
        return RedirectToAction(nameof(Index));
    }
}
```

⚠️ **In de Edit-view** moet de primary key mee in het formulier: `<input type="hidden" asp-for="VoertuigId" />`

Delete-view (formulier):

```
<form asp-action="Delete">
    <input type="hidden" asp-for="VoertuigId" />
    <input type="submit" value="Verwijderen" class="btn btn-danger" />
</form>
```

Links in de Index-tabel:

```
<a asp-action="Edit" asp-route-id="@item.VoertuigId">Edit</a> |
<a asp-action="Details" asp-route-id="@item.VoertuigId">Details</a> |
<a asp-action="Delete" asp-route-id="@item.VoertuigId">Delete</a>
```

Views genereren: rechtsklik in de actie → Add View → Razor View → Template **Details / Edit / Delete** → Model class.

`[Bind]` (scaffolding gebruikt dit) laat alleen de genoemde velden binnen: `Create([Bind("VoertuigId,Merk,MerkType")] Voertuig voertuig)`.

---

10. Relaties + Include

1-n: één garage heeft veel voertuigen

```
public class Garage
{
    public int GarageId { get; set; }
    public string Naam { get; set; } = string.Empty;
    public List<Voertuig> Voertuigen { get; set; } = new();   // collection navigation
}

public class Voertuig
{
    public int VoertuigId { get; set; }
    public int GarageId { get; set; }          // foreign key
    public Garage? Garage { get; set; }        // reference navigation
}
```

n-m: een voertuig heeft veel opties, een optie zit in veel voertuigen

EF maakt de tussentabel zelf aan.

```
public class Optie
{
    public int OptieId { get; set; }
    public string Naam { get; set; } = string.Empty;
    public List<Voertuig> Voertuigen { get; set; } = new();
}
// in Voertuig:
public List<Optie> Opties { get; set; } = new();
```

1-1: een voertuig heeft één keuringsrapport

```
public class Keuring
{
    public int KeuringId { get; set; }
    public DateTime Datum { get; set; }
    public int VoertuigId { get; set; }         // FK aan de "afhankelijke" kant
    public Voertuig? Voertuig { get; set; }
}
// in Voertuig:
public Keuring? Keuring { get; set; }
```

Gerelateerde data ophalen

```
_context.Voertuigen.Include(v => v.Garage).ToList();                    // 1 niveau
_context.Garages.Include(g => g.Voertuigen).ThenInclude(v => v.Opties).ToList(); // 2 niveaus
_context.Voertuigen.Include(v => v.Garage).Include(v => v.Opties).ToList();      // meerdere
```

Zonder `Include` is `v.Garage` **null**. In de view schrijf je dan `@item.Garage?.Naam`.

Dropdown voor de foreign key (Create/Edit)

```
// GET én POST (vóór return View):
ViewData["GarageId"] = new SelectList(_context.Garages, "GarageId", "Naam", voertuig?.GarageId);
```

```
<select asp-for="GarageId" asp-items="ViewBag.GarageId" class="form-select">
    <option value="">-- Kies een garage --</option>
</select>
```

⚠️ Bij POST valideert ASP.NET de navigation property `Garage` niet zolang die **nullable** is (`Garage?`). Laat die `?` dus staan.

---

11. EF Configuration

Met annotaties

Annotatie	Doet
`[Key]`	primary key (als de naam niet `Id` / `KlasseId` is)
`[DatabaseGenerated(DatabaseGeneratedOption.None)]`	key niet automatisch nummeren
`[Table("tblVoertuig")]` (boven klasse)	tabelnaam
`[Column("Prijs", TypeName = "decimal(8,2)")]`	kolomnaam / type
`[ForeignKey("GarageId")]` (boven navigation)	foreign key expliciet
`[NotMapped]`	geen kolom in de DB (bv. berekende property)
`[MaxLength(100)]` / `[StringLength(100)]`	nvarchar(100)
`[Required]`	NOT NULL
`[Index(nameof(MerkType), IsUnique = true)]` (boven klasse, `using Microsoft.EntityFrameworkCore;`)	unieke index
`[Precision(8, 2)]`	decimal(8,2)

Met Fluent API (in de DbContext)

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);

    modelBuilder.Entity<Voertuig>(e =>
    {
        e.ToTable("Voertuigen");
        e.HasKey(v => v.VoertuigId);
        e.Property(v => v.Merk).IsRequired().HasMaxLength(100);
        e.Property(v => v.VerkoopPrijs).HasColumnType("decimal(8,2)");
        e.Property(v => v.AanwezigInShowroom).HasDefaultValue(false);
        e.HasIndex(v => v.MerkType).IsUnique();
        e.Ignore(v => v.Omschrijving);                    // = [NotMapped]

        e.HasOne(v => v.Garage)                           // 1-n
         .WithMany(g => g.Voertuigen)
         .HasForeignKey(v => v.GarageId)
         .OnDelete(DeleteBehavior.Restrict);              // Cascade / Restrict / SetNull
    });

    modelBuilder.Entity<Voertuig>()                       // n-m
        .HasMany(v => v.Opties)
        .WithMany(o => o.Voertuigen);

    modelBuilder.Entity<Voertuig>()                       // 1-1
        .HasOne(v => v.Keuring)
        .WithOne(k => k.Voertuig)
        .HasForeignKey<Keuring>(k => k.VoertuigId);

    // Startdata (komt in de migration; Id's VERPLICHT, ook de FK's)
    modelBuilder.Entity<Garage>().HasData(
        new Garage { GarageId = 1, Naam = "Garage Hasselt" });
    modelBuilder.Entity<Voertuig>().HasData(
        new Voertuig { VoertuigId = 1, Merk = "AUDI", MerkType = "A4", Bouwjaar = 2019, Km = 85000, VerkoopPrijs =
24950m, GarageId = 1 });
}
```

Configuratie in een aparte klasse

```
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;

public class VoertuigConfiguration : IEntityTypeConfiguration<Voertuig>
{
    public void Configure(EntityTypeBuilder<Voertuig> builder)
    {
        builder.Property(v => v.Merk).IsRequired().HasMaxLength(100);
        builder.Property(v => v.VerkoopPrijs).HasColumnType("decimal(8,2)");
    }
}
// In OnModelCreating:
modelBuilder.ApplyConfiguration(new VoertuigConfiguration());
// of alle configuraties in het project in één keer:
modelBuilder.ApplyConfigurationsFromAssembly(typeof(VoertuigDbContext).Assembly);
```

Na **elke** wijziging aan het model of de configuratie: `Add-Migration Naam` + `Update-Database`.

---

12. EF Tools

Package Manager Console (Tools → NuGet Package Manager → Package Manager Console) | Commando | Doet | |---|---| | `Add-Migration Initial` | nieuwe migration (bouwplan) maken | | `Update-Database` | alle migrations uitvoeren | | `Update-Database Initial` | terug naar (of tot) die migration | | `Update-Database 0` | alle migrations ongedaan maken (lege DB) | | `Remove-Migration` | laatste migration verwijderen (nog niet toegepast) | | `Get-Migration` | lijst van migrations | | `Drop-Database` | databank verwijderen | | `Script-Migration` | SQL-script van de migrations |

Database-first (classes laten genereren uit een bestaande databank):

```
Scaffold-DbContext "Server=(localdb)\MSSQLLocalDB;Database=VoertuigDb;Trusted_Connection=True"
Microsoft.EntityFrameworkCore.SqlServer -OutputDir Models -Context VoertuigDbContext
```

(In de PMC schrijf je **één** backslash, want dat is geen JSON.)

.NET CLI (terminal, alternatief):

```
dotnet tool install --global dotnet-ef
dotnet ef migrations add Initial
dotnet ef database update
dotnet ef migrations remove
```

Een migration bestaat uit `Up()` (wijziging doorvoeren) en `Down()` (ongedaan maken). Ze staan in de map `Migrations/`, samen met een `...ModelSnapshot.cs`.

---

13. Scaffolding in Visual Studio

Volledige controller + alle views: rechtsklik op `Controllers` → Add → **New Scaffolded Item** (of Add → Controller) → **MVC Controller with views, using Entity Framework** →

- Model class: `Voertuig`
- DbContext class: `VoertuigDbContext`
- Generate views ✔, Use a layout page ✔ → controllernaam → Add

Dit genereert Index, Details, Create, Edit en Delete, met views in `Views/Voertuig/`.

Eén view: rechtsklik in een actie → Add View → **Razor View** → Template: | Template | Model | Voor | |---|---|---| | List | `IEnumerable<Voertuig>` | tabel met alle items | | Details | `Voertuig` | één item tonen | | Create | `Voertuig` | leeg formulier | | Edit | `Voertuig` | ingevuld formulier | | Delete | `Voertuig` | bevestigen | | Empty | – | lege view |

Scaffolding installeert soms zelf het package `Microsoft.VisualStudio.Web.CodeGeneration.Design`. Dat is normaal. Duurt het lang of crasht het? Bouw eerst het project (Ctrl+Shift+B) en probeer opnieuw.

---

14. Partial View

- Bestandsnaam met `_` vooraan, bv. `Views/Shared/_VoertuigCard.cshtml`, met `@model Voertuig` bovenaan.
- ASP.NET zoekt eerst in `Views/<Controller>/`, daarna in `Views/Shared/`.

Aanroepen	Code
Partial TagHelper (aanbevolen)	`<partial name="_VoertuigCard" model="voertuig" />`
HTML-helper	`@await Html.PartialAsync("_VoertuigCard", voertuig)`
Met extra data	`<partial name="_VoertuigCard" model="voertuig" view-data="ViewData" />`
Vanuit een controller	`return PartialView("_VoertuigCard", voertuig);`

Typisch gebruik: een kaartje per item in een lijst, een stuk formulier dat je in Create **en** Edit gebruikt, of een melding.

---

15. TagHelpers

Basis

```
using Microsoft.AspNetCore.Razor.TagHelpers;

public class CarInfoTagHelper : TagHelper              // <car-info aantal="5"></car-info>
{
    public int Aantal { get; set; }

    public override void Process(TagHelperContext context, TagHelperOutput output)
    {
        output.TagName = "div";
        output.TagMode = TagMode.StartTagAndEndTag;
        output.Attributes.SetAttribute("class", "btn btn-outline-primary");
        output.Content.SetHtmlContent($"Aantal voertuigen <span class=\"badge bg-primary\">{Aantal}</span>");
    }
}
```

Verplicht in `_ViewImports.cshtml`: `@addTagHelper *, <ProjectNaam>`

Naamregels en attributen

C#	HTML
`CarInfoTagHelper`	`<car-info>`
`public int Aantal`	`aantal="5"`
`public string BadgeKleur`	`badge-kleur="danger"`
`[HtmlAttributeName("kleur")] public string BadgeKleur`	`kleur="danger"`
`[HtmlTargetElement("auto")]`	`<auto>`
`[HtmlTargetElement("a", Attributes = "knop-kleur")]`	bestaande `<a knop-kleur="...">`
`[HtmlTargetElement(Attributes = "toon-als")]`	**elk** element met dat attribuut
`[HtmlTargetElement("bold")]` + `[HtmlTargetElement(Attributes = "bold")]`	`<bold>` én `<p bold>`

Wat kan je met output doen?

Code	Effect
`output.TagName = "span";`	ander HTML-element
`output.TagName = null;`	alleen de inhoud, zonder omhullende tag
`output.Attributes.SetAttribute("class", "x");`	attribuut zetten/vervangen
`output.Attributes.RemoveAll("bold");`	attribuut weghalen
`output.Content.SetContent("tekst")`	inhoud (HTML wordt **ge-escapet**)
`output.Content.SetHtmlContent("<b>x</b>")`	inhoud als HTML
`output.PreContent.SetHtmlContent("<b>")` / `output.PostContent.SetHtmlContent("</b>")`	vóór/na de bestaande inhoud
`output.PreElement.SetHtmlContent("<hr/>")` / `output.PostElement`	vóór/na de hele tag
`output.SuppressOutput();`	niets tonen
`output.TagMode = TagMode.StartTagAndEndTag;`	**verplicht** als je de tag zelfsluitend schrijft (`<car-info />`), anders wordt de inhoud niet getoond

De inhoud tussen de tags gebruiken (ProcessAsync)

```
// <titel>Mijn voertuigen</titel>  ->  <h1 class="text-primary">MIJN VOERTUIGEN</h1>
public class TitelTagHelper : TagHelper
{
    public override async Task ProcessAsync(TagHelperContext context, TagHelperOutput output)
    {
        var inhoud = (await output.GetChildContentAsync()).GetContent();
        output.TagName = "h1";
        output.Attributes.SetAttribute("class", "text-primary");
        output.Content.SetContent(inhoud.ToUpper());
    }
}
```

Iets tonen op voorwaarde

```
// <div toon-als="@(Model.Km > 100000)">Veel km!</div>
[HtmlTargetElement(Attributes = "toon-als")]
public class ToonAlsTagHelper : TagHelper
{
    public bool ToonAls { get; set; }
    public override void Process(TagHelperContext context, TagHelperOutput output)
    {
        if (!ToonAls) output.SuppressOutput();
        output.Attributes.RemoveAll("toon-als");
    }
}
```

Met TagBuilder

```
using Microsoft.AspNetCore.Mvc.Rendering;

TagBuilder h4 = new TagBuilder("h4");
h4.AddCssClass("card-title");
h4.InnerHtml.Append(Voertuig.Merk);                // tekst (veilig)
TagBuilder body = new TagBuilder("div");
body.AddCssClass("card-body");
body.InnerHtml.AppendHtml(h4);                      // andere TagBuilder erin

output.TagName = "div";
output.Attributes.SetAttribute("class", "card");
output.Content.SetHtmlContent(body);
```

Databank of huidige pagina gebruiken

```
using Microsoft.AspNetCore.Mvc.Rendering;
using Microsoft.AspNetCore.Mvc.ViewFeatures;

public class AantalVoertuigenTagHelper : TagHelper     // <aantal-voertuigen />
{
    private readonly VoertuigDbContext _context;
    public AantalVoertuigenTagHelper(VoertuigDbContext context) { _context = context; }  // DI werkt hier ook

    [ViewContext, HtmlAttributeNotBound]
    public ViewContext ViewContext { get; set; } = null!;  // info over de huidige request

    public override void Process(TagHelperContext context, TagHelperOutput output)
    {
        var controller = ViewContext.RouteData.Values["controller"]?.ToString();
        output.TagName = "span";
        output.TagMode = TagMode.StartTagAndEndTag;   // nodig bij <aantal-voertuigen />, anders blijft het leeg!
        output.Content.SetContent($"{_context.Voertuigen.Count()} voertuigen (pagina: {controller})");
    }
}
```

Ingebouwde TagHelpers

TagHelper	Voorbeeld
Link	`<a asp-controller="Voertuig" asp-action="Edit" asp-route-id="5">`
Formulier	`<form asp-action="Create" method="post">` (voegt een anti-forgery token toe)
Label / input / textarea	`<label asp-for="Merk">`, `<input asp-for="Merk">`, `<textarea asp-for="Omschrijving">`
Dropdown	`<select asp-for="Merk" asp-items="ViewBag.Merken">`
Validatie	`<span asp-validation-for="Merk">`, `<div asp-validation-summary="All">`
Partial	`<partial name="_X" model="m" />`
ViewComponent	`<vc:header-menu></vc:header-menu>`
Omgeving	`<environment include="Development">...</environment>`
Cache-busting	`<link href="~/css/site.css" asp-append-version="true" />`
Afbeelding	`<img src="~/img/auto.png" asp-append-version="true" />`

`<input asp-for>` kiest zelf het type: `int` / `decimal` geeft `number`, `bool` geeft `checkbox`, `DateTime` geeft `datetime-local`, en `[DataType(DataType.Date)]` geeft `date`.

---

16. ViewComponents

```
public class HeaderMenuViewComponent : ViewComponent
{
    private readonly VoertuigDbContext _context;
    public HeaderMenuViewComponent(VoertuigDbContext context) { _context = context; }

    public IViewComponentResult Invoke()
    {
        var merken = _context.Voertuigen.Select(v => v.Merk).Distinct().OrderBy(m => m).ToList();
        return View(merken);                    // -> Default.cshtml
    }
}
```

View-locatie: `Views/Shared/Components/HeaderMenu/Default.cshtml` (of `Views/<Controller>/Components/HeaderMenu/Default.cshtml`)

Wat	Code
Aanroepen	`@await Component.InvokeAsync("HeaderMenu")`
Als tag	`<vc:header-menu></vc:header-menu>` (vereist `@addTagHelper *, <ProjectNaam>`)
Met parameters	`@await Component.InvokeAsync("VoertuigLijst", new { merk = "BMW", aantal = 3 })`
Als tag met parameters	`<vc:voertuig-lijst merk="BMW" aantal="3"></vc:voertuig-lijst>`
Andere view gebruiken	`return View("Compact", merken);` → `.../HeaderMenu/Compact.cshtml`
Alleen tekst	`return Content("Geen voertuigen");`
Vanuit een controller	`return ViewComponent("HeaderMenu");`

Async + parameters:

```
public async Task<IViewComponentResult> InvokeAsync(string merk, int aantal = 5)
{
    var lijst = await _context.Voertuigen
        .Where(v => v.Merk == merk)
        .OrderByDescending(v => v.Bouwjaar)
        .Take(aantal)
        .ToListAsync();
    return View(lijst);
}
```

Beschikbaar in een ViewComponent: `Request.Query["merk"]`, `RouteData.Values["id"]`, `HttpContext`, `ViewBag`, `ViewData`, `User`

Het actieve menu-item markeren:

```
ViewBag.Actief = Request.Query["merk"].ToString();
```

```
<a class="btn @(merk == ViewBag.Actief ? "btn-warning" : "btn-info") m-1" asp-controller="Voertuig" asp-action="Index"
asp-route-merk="@merk">@merk</a>
```

---

17. LINQ queries

Alle voorbeelden werken op `_context.Voertuigen` (een databank) **en** op een gewone `List<Voertuig>`. Bij een databank zet je `.ToList()` (of `await ...ToListAsync()`) op het einde.

Filteren en sorteren

```
.Where(v => v.Merk == "BMW")
.Where(v => v.Km < 100000 && v.AanwezigInShowroom)
.Where(v => v.Merk == "BMW" || v.Merk == "AUDI")
.Where(v => v.Merk.Contains(zoek))                // bevat
.Where(v => v.Merk.StartsWith("P"))               // begint met
.Where(v => merken.Contains(v.Merk))              // zit in een lijst (List<string> merken)
.Where(v => v.Bouwjaar >= 2020)
.OrderBy(v => v.Merk).ThenBy(v => v.MerkType)
.OrderByDescending(v => v.VerkoopPrijs).ThenByDescending(v => v.Bouwjaar)
```

Projectie (kiezen wat je terugkrijgt)

```
.Select(v => v.Merk)                                          // alleen merken
.Select(v => v.Merk).Distinct()                               // unieke merken
.Select(v => new VoertuigKort { Naam = v.Merk + " " + v.MerkType }) // naar een ViewModel
```

Eén element

Methode	Als er niets is	Als er meerdere zijn
`First(...)`	exception	eerste
`FirstOrDefault(...)`	`null`	eerste
`Single(...)`	exception	exception
`SingleOrDefault(...)`	`null`	exception
`Last` / `LastOrDefault` (vereist OrderBy)	idem	laatste

Tellen en berekenen

```
.Count()                        .Count(v => v.AanwezigInShowroom)
.Sum(v => v.VerkoopPrijs)       .Average(v => v.VerkoopPrijs)
.Min(v => v.Bouwjaar)           .Max(v => v.Km)
.Any()                          .Any(v => v.Km > 200000)       // minstens één?
.All(v => v.Km >= 0)                                           // allemaal?
```

⚠️ `Average`, `Min` en `Max` op een **lege** lijst geven een exception. Controleer eerst met `.Any()`.

Delen

```
.Take(3)                                // eerste 3
.Skip(3)                                // eerste 3 overslaan
.Skip((pagina - 1) * grootte).Take(grootte)   // paging
.OrderByDescending(v => v.VerkoopPrijs).First()   // de duurste
```

Groeperen

```
var perMerk = _context.Voertuigen
    .GroupBy(v => v.Merk)
    .Select(g => new MerkStatistiek
    {
        Merk = g.Key,
        Aantal = g.Count(),
        GemiddeldePrijs = g.Average(v => v.VerkoopPrijs),
        Duurste = g.Max(v => v.VerkoopPrijs)
    })
    .OrderByDescending(m => m.Aantal)
    .ToList();
```

```
@model IEnumerable<MerkStatistiek>
<table class="table">
    <tr><th>Merk</th><th>Aantal</th><th>Gemiddelde prijs</th></tr>
    @foreach (var m in Model)
    {
        <tr><td>@m.Merk</td><td>@m.Aantal</td><td>@m.GemiddeldePrijs.ToString("C")</td></tr>
    }
</table>
```

Join (zonder navigation properties)

```
var lijst = _context.Voertuigen
    .Join(_context.Garages,
          v => v.GarageId,          // sleutel links
          g => g.GarageId,          // sleutel rechts
          (v, g) => new { v.Merk, v.MerkType, GarageNaam = g.Naam })
    .ToList();
```

Met navigation properties is dit eenvoudiger: `.Include(v => v.Garage)` of `.Select(v => v.Garage!.Naam)`.

SelectMany (lijsten in lijsten plat maken)

```
var alleOpties = _context.Voertuigen.SelectMany(v => v.Opties).Select(o => o.Naam).Distinct().ToList();
```

Query-syntax (zelfde resultaat, andere schrijfwijze)

```
var bmws = from v in _context.Voertuigen
           where v.Merk == "BMW" && v.Km < 100000
           orderby v.VerkoopPrijs descending
           select v;

var perMerk = from v in _context.Voertuigen
              group v by v.Merk into g
              select new { Merk = g.Key, Aantal = g.Count() };

var metGarage = from v in _context.Voertuigen
                join g in _context.Garages on v.GarageId equals g.GarageId
                select new { v.Merk, g.Naam };
```

Filteren in een controller (zoeken + sorteren)

```
public IActionResult Index(string? zoek, string? sorteer)
{
    var query = _context.Voertuigen.AsQueryable();

    if (!string.IsNullOrEmpty(zoek))
        query = query.Where(v => v.Merk.Contains(zoek) || v.MerkType.Contains(zoek));

    query = sorteer switch
    {
        "prijs"      => query.OrderBy(v => v.VerkoopPrijs),
        "prijs_desc" => query.OrderByDescending(v => v.VerkoopPrijs),
        "km"         => query.OrderBy(v => v.Km),
        _            => query.OrderBy(v => v.Merk)
    };

    ViewBag.Zoek = zoek;
    return View(query.ToList());
}
```

```
<a asp-action="Index" asp-route-sorteer="prijs" asp-route-zoek="@ViewBag.Zoek">Prijs ↑</a>
<a asp-action="Index" asp-route-sorteer="prijs_desc" asp-route-zoek="@ViewBag.Zoek">Prijs ↓</a>
```

---

18. Formulieren: dropdown, enum, checkbox, datum

Dropdown met vaste waarden:

```
ViewBag.Merken = new SelectList(new[] { "AUDI", "BMW", "PORSCHE" });
```

```
<select asp-for="Merk" asp-items="ViewBag.Merken" class="form-select"></select>
```

Dropdown uit de databank (unieke waarden):

```
ViewBag.Merken = new SelectList(_context.Voertuigen.Select(v => v.Merk).Distinct().ToList());
```

Enum:

```
public enum Brandstof { Benzine, Diesel, Elektrisch, Hybride }
// in het model:
public Brandstof Brandstof { get; set; }
```

```
<select asp-for="Brandstof" asp-items="Html.GetEnumSelectList<Brandstof>()" class="form-select"></select>
```

(Een enum wordt in de databank opgeslagen als een int.)

Checkbox (bool):

```
<div class="form-check">
    <input asp-for="AanwezigInShowroom" class="form-check-input" />
    <label asp-for="AanwezigInShowroom" class="form-check-label"></label>
</div>
```

Datum:

```
[DataType(DataType.Date)]
[DisplayFormat(DataFormatString = "{0:dd/MM/yyyy}")]
public DateTime AankoopDatum { get; set; }
```

```
<input asp-for="AankoopDatum" class="form-control" />   @* wordt type="date" *@
@Model.AankoopDatum.ToString("dd/MM/yyyy")
```

Radio buttons:

```
@foreach (var b in Enum.GetValues<Brandstof>())
{
    <input type="radio" asp-for="Brandstof" value="@b" /> @b
}
```

Melding na opslaan (TempData + Bootstrap-alert):

```
TempData["Melding"] = "Voertuig opgeslagen!";
return RedirectToAction(nameof(Index));
```

```
@if (TempData["Melding"] != null)
{
    <div class="alert alert-success">@TempData["Melding"]</div>
}
```

Bootstrap-classes die vaak terugkomen

Wat	Classes
Knoppen	`btn btn-primary`, `btn-secondary`, `btn-success`, `btn-danger`, `btn-warning`, `btn-info`, `btn-outline-primary`, `btn-sm`
Badge	`badge bg-primary` (`bg-success`, `bg-danger`, …)
Kaart	`card`, `card-body`, `card-title`, `card-text`, `card-header`
Tabel	`table table-striped table-hover`
Formulier	`form-control`, `form-select`, `form-check`, `form-check-input`, `form-label`
Melding	`alert alert-success` / `alert-danger`
Spatie	`m-1` … `m-5` (margin), `p-1` … `p-5` (padding), `mt-3` (margin-top), `mb-2`
Tekst	`text-center`, `text-danger`, `text-muted`, `fw-bold`
Grid	`container`, `row`, `col-md-4`
Menu	`navbar navbar-expand-sm bg-primary navbar-dark`